Hasini Sandunika Silva
Jun 7, 2021 · 6 min read · Listen

Design Patterns for Microservice Architecture

Design Patterns for Microservice Architecture

Before we talk about the design patterns for microservices we need to understand the basic key features which have been contributed to building the microservices architecture. Here I will discuss some of the design patterns currently used by the developers to build microservices.

Aggregator Pattern

This can be applied in 3 different ways such as parallel, chained, and branch pattern.

Let's understand each by using the following example.

Assume there exist 3 services such as student registration, exams, courses allocation, and suppose a consumer called the student management system consumes all these 3 services. Suppose the student registration service sends the response as the id of the created student. So, this will be used to get, set exam results, and allocate courses for the particular student by id. Assume a particular front-end web page of the student management system needs to display the results with the descriptions of allocated courses by student id. Refer to the following figure 1.

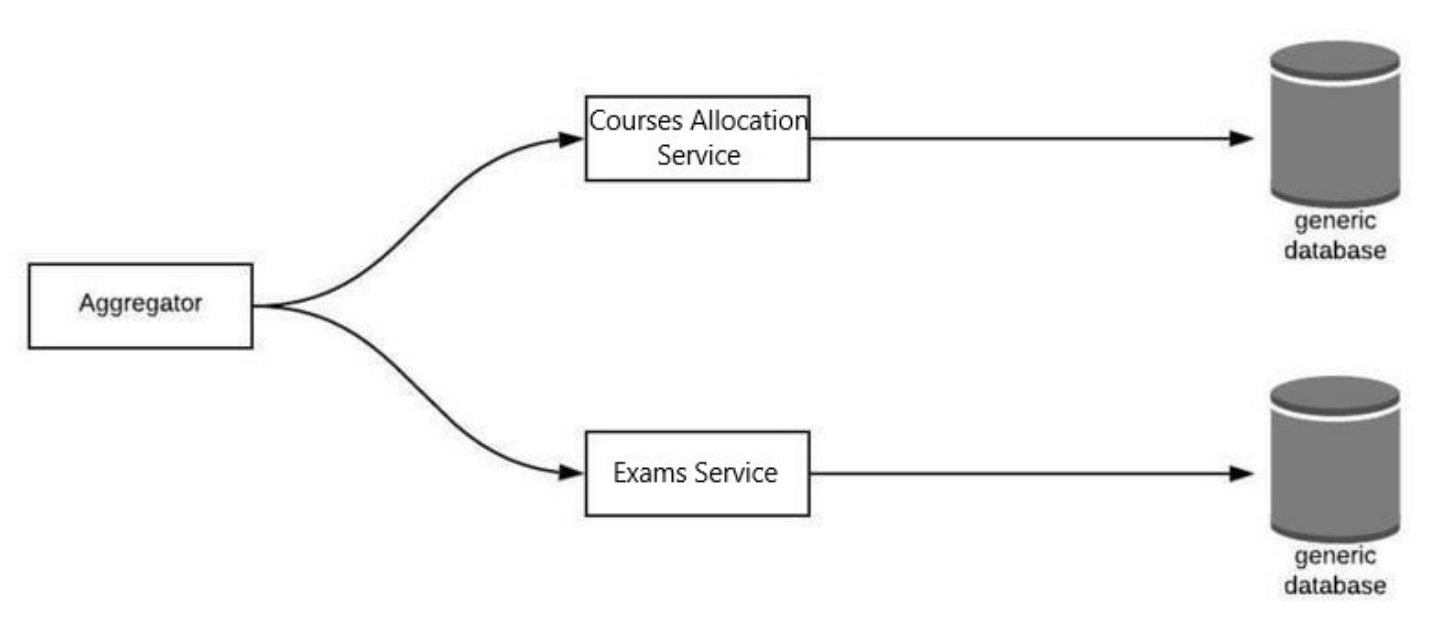


Figure 1. Aggregator Pattern.

The following defines how we can apply each pattern to obtain the desired output.

Parallel or Scatter Gather Pattern

Here, this is implemented by creating a single service by aggregating both exams and courses allocation services. Here the aggregated service sends the student id to both services parallelly and aggregates the responses into a single response and sends that to the front-end.

Here, we call other services inside the aggregate service parallelly.

Chained Pattern

Here also we can implement the chained pattern by aggregating the above 2 services into a single service. Inside the aggregated service, first, it calls for the courses allocation service and obtains the allocated courses of the particular student by id. Here assume we can obtain the exam results by passing the allocated courses with the student id. After that, the new service aggregates the responses of both services into a single response and sends that to the front-end.

Usually, in the chained pattern, we call the services inside the aggregated service one after the other.

Branch Pattern

Here also we can implement the branching pattern by aggregating exams and courses allocation services into a single service. Assume the front-end only needs to display the exam results of students, who do not hold any repeated courses. Here, the aggregated service first calls for the courses allocation service and obtains the allocated courses with descriptions of the particular student by id. But, if this student does not have any repeated courses, then it calls for the exams service and aggregates the responses, and sends it to the front-end. Otherwise, the new service does not call for the exams service.

Here inside the aggregate service, we decide the next service call based on the response received from the previous service.

Circuit Breaker Pattern

Before talking about the circuit breaker pattern, it is better to understand the causes of failing the service.

Assume, a client accesses the web application through a web server. Here, the web server allocates a single thread per request to call the back-end or the corresponding service that needs the response. Refer to the following figure 2.

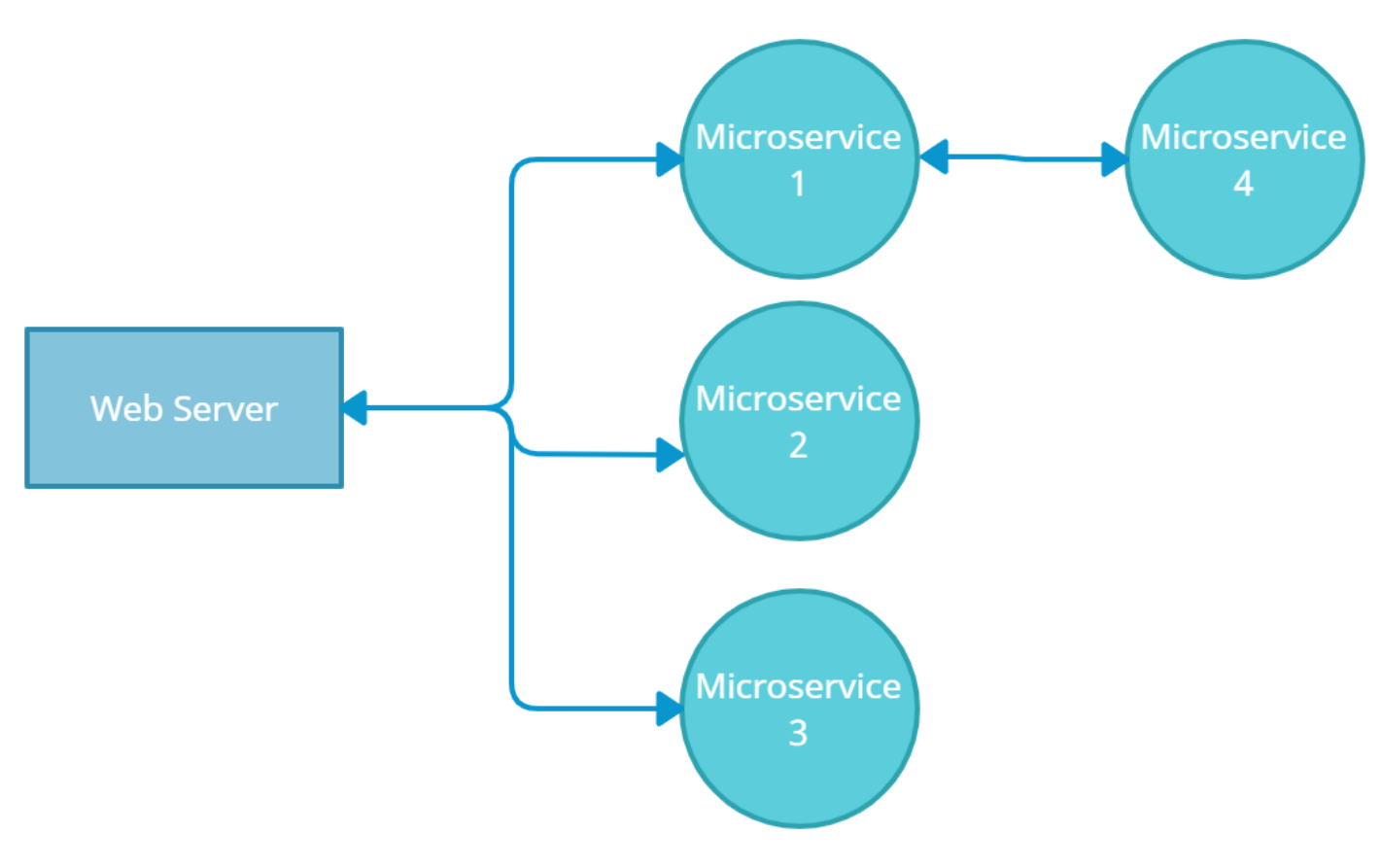


Figure 2. Application Architecture.

Assume, the web server needs to obtain dynamic content by accessing through the path microservices 1 and 4. But assume that service 4 takes much time to give the response to service 1. Because of that, service 1 has to wait for some time to get the response from service 4. This indicates that service 1 is in a cascade queue. Here the corresponding thread created for this particular request has to wait in the thread pool until the timeout happens or response reaches. Assume, this gets more than one request from the server. So, all threads created per request have to wait in a thread pool. As a solution for that, we can use the circuit breaker pattern.

Usually, we apply the circuit breaker pattern in such a way that, if the time taken to respond to a particular request exceeds the threshold limit then it sends an error message to the consumer while not letting to send new requests to the service again and again (the request let others know that the service is failing using a flag because of that, other requests will not reach to the service and go back). The following figure 3 illustrates an application of the circuit breaker pattern.

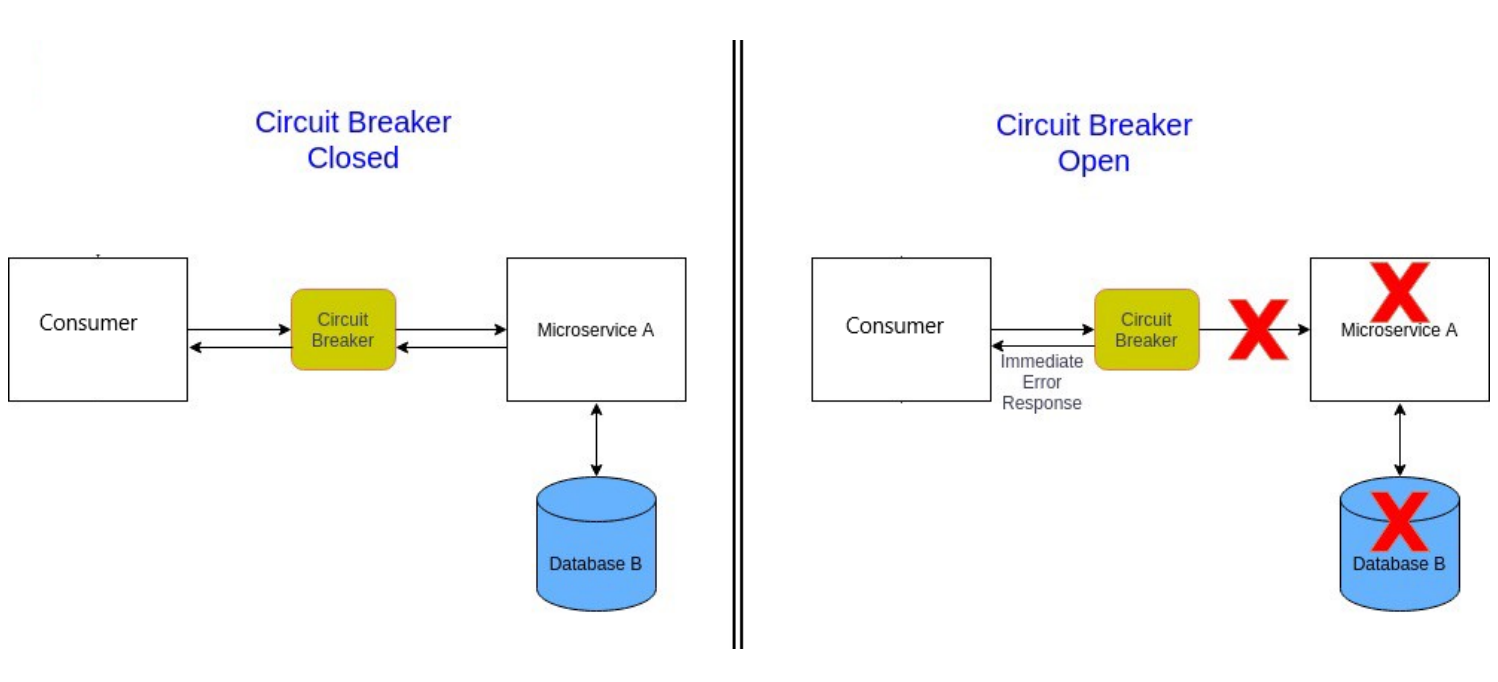


Figure 3. Circuit Breaker Pattern.

So, let's understand what is happening inside that.

Assume the threshold timeout value is 200ms. Here, if during a certain period, more than 75% of requests take around 150ms-200ms to send the response to the consumer then, it is known that service A is falling slowly. But, if some requests exceed the threshold limit of 200ms then, the consumer identifies that service A is not responding anymore. So, next the circuit breaker comes into the picture. Here, the circuit breaker breaks the connection between the consumer and service A. Because of that, the request of the consumer will not be sent to service A. Here, no queue is created because after breaking the connection between the consumer and service A, the new requests cannot reach the service.

To establish the connection back during this break time, in the background a ping is sent to service A from time to time. If again the time taken to get the response from service A gets normal, the connection will be opened again to the consumer.

Proxy Pattern

When developers use the agile approach in the software development process, sometimes they have to change the features of the services that are already deployed. Assume as an example, in a student management system currently use the student code, but as per the new requirement, this attribute has to be changed as the student id which has only integers. To allow that, we can use the proxy design pattern. The following figure 4 defines the implementation of the proxy design pattern.

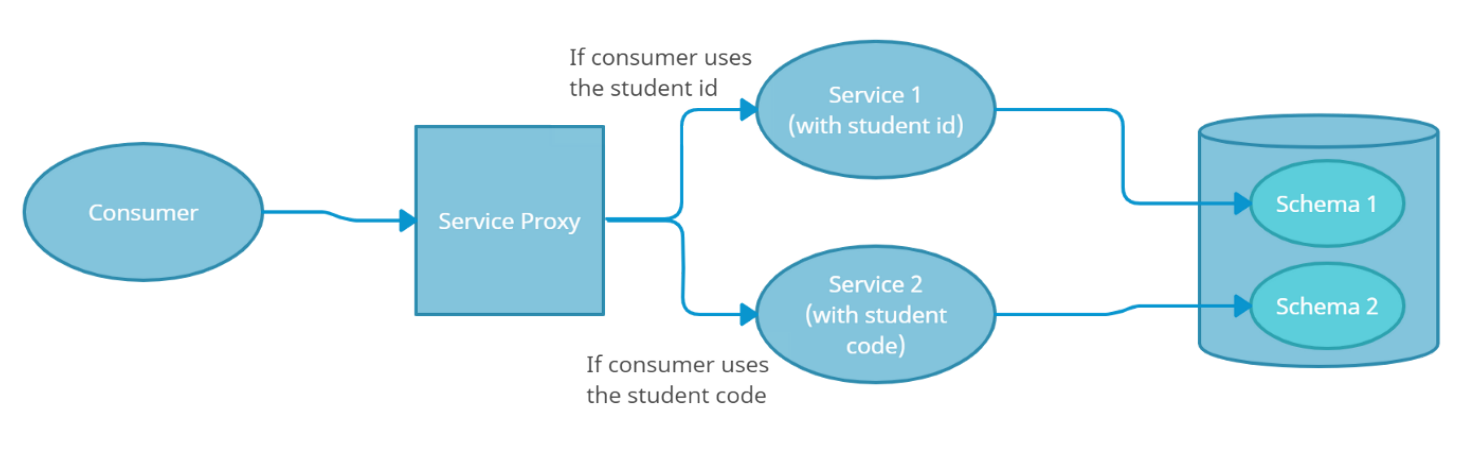

If consumer uses the student id
Service 1 (with student id)
Consumer
Service Proxy
Service 2 (with student code)
If consumer uses the student code
Schema 1
Schema 2

Figure 4. Implementation of the Proxy Pattern.

As per the above illustration, developers can create a new proxy service between the consumer and the new, existing services. If the consumer accesses the service with the student id, the proxy service will call for service 1 by obtaining the IP address of service 1 from the service discovery. Otherwise, if the consumer needs to access the service with student code, the proxy service will call for service 2 by obtaining the IP address of service 2 from the service discovery. Here it is important to use a service discovery mechanism to discover the next service to call because, when developers deploy service 1, a new IP is added for service 1, and also because of using microservices it is appropriate to follow the basic principles such as independence. To implement that developers can use the WSO2 governor registry, consul, etc. After all the consumers have shifted to service 1, developers can shut down service 2 because hereafter there is no use of it.

When using the proxy design pattern, it's important to maintain a proper semantic versioning approach to let others know about the changes that have been done to the service. When we use this pattern there may occur cascade failures because the proxy is calling to another service via the service discovery, again the next service also may call for another service via the service discovery. Assume the final service takes much time to send the response back to the previous service. Here, the other services need to wait to get the responses from the next service they called. To avoid this situation developers can use message queues. Message queues store the messages received and process them one by one. Refer to the following figure 5.

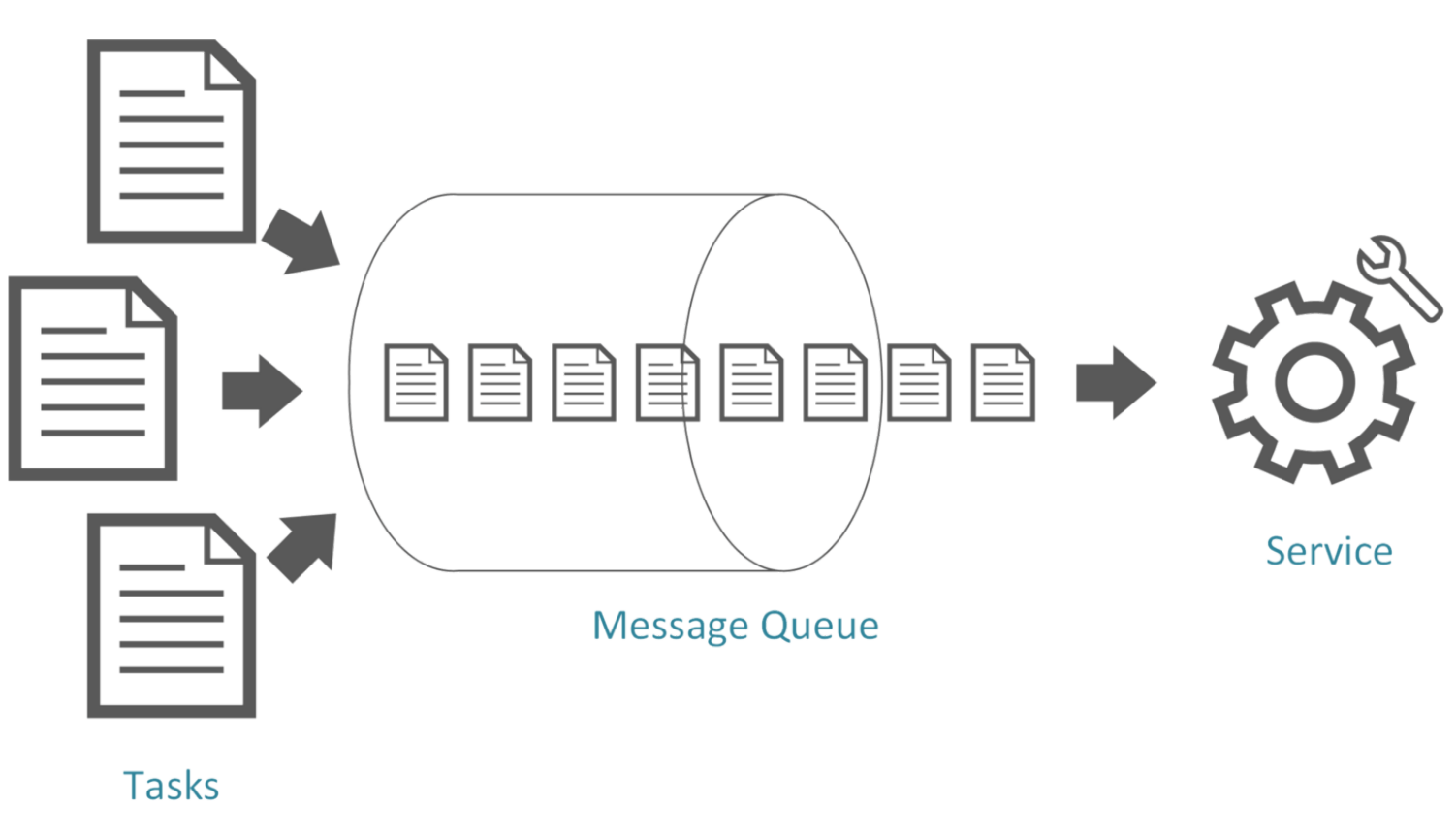


Figure 5. The structure of the Message Queue.

But sometimes message queues may fail. To tackle these kinds of situations, developers need to use some redundant mechanisms when using these approaches.

References

- https://www.youtube.com/watch?v=DZeXqbFvYNA&list=PLD-mYtebG3X9HaZ1T39-aF4ghEtWy9-v3&index=5
- https://www.youtube.com/watch?v=mw8W-RpvCu0&list=PLD-mYtebG3X9HaZ1T39-aF4ghEtWy9-v3&index=6
- https://www.youtube.com/watch?v=7mBvb7nKONg&list=PLD-mYtebG3X9HaZ1T39-aF4ghEtWy9-v3&index=7
- https://towardsdatascience.com/microservice-architecture-and-its-10-most-important-design-patterns-824952d7fa41

19

More from Hasini Sandunika Silva

BSc (Hons) in Computer Science

Follow

Jun 7, 2021

An Introduction Microservices and Their Best Practices

Early and nowadays developers use monolithic architecture to build their web-based applications. But there are some pros and cons related to this.…

Microservices · 4 min read

Share your ideas with millions of readers. Write on Medium

Published in **Nerd For Tech** · Jun 4, 2021

An Introduction to Dockers and Containers

Early days, developers used 2 separate physical servers such as web and application to deploy their web applications. But here, all memory and other features may not be used totally which implies this is not 100%…

Containers · 3 min read

Published in **Nerd For Tech** · May 31, 2021

Things You Must Know About the Modern JavaScript

Overview JavaScript or JS is a multi-paradigm language and this supports both object-oriented and functional programming. This was improved and released several editions after the first release. JavaScript holds different…

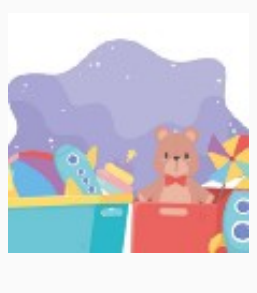
Modern Javascript · 3 min read

Published in **CodeX** · May 26, 2021

An Introduction to Memento Design Pattern

The memento design pattern allows to undo or restore the previous version of the object. This is categorized under the behavioral pattern. As per the book written by the gang of four (GOF), there are 3 pillars in the memento…

Design Patterns · 4 min read

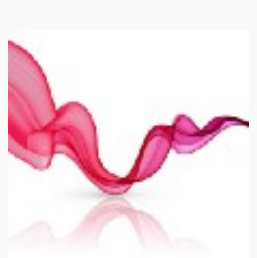


Published in **Nerd For Tech** · May 25, 2021

An Introduction to Chain of Responsibility Design Pattern

The chain of responsibility pattern can be applied when we need to apply the loosely-coupling concept in our work. This is categorized under the…

Chain Of Responsibility · 3 min read



Love podcasts or audiobooks? Learn on the go with our new app. Try Knowable

Get unlimited access

Search

Hasini Sandunika Silva
55 Followers
BSc (Hons) in Computer Science
Follow

More from Medium

mohammad hosein abbasi
Microservices Versus APIs

Jason Byrne
What is Pub/Sub event-driven architecture?

Alberto Basalo
Starting a software project from The Domain

Oleksii Dushenin
How To Understand Microservices Architecture

Help Status Writers Blog Careers Privacy Terms About Knowable